

Problem-Sized Lean Worlds

Persistent, checkable research records for AI-assisted mathematics

WILL COOK*

September 2026

ABSTRACT

Frontier models now produce candidate proofs of research problems faster than mathematicians can read them. What is rarely preserved is the work around a proof: the reasoning stays in a session, failed routes are discarded, and the result reaches the public without the explanation, attribution and sense of difficulty that make it usable. This paper describes a research environment organised around a persistent unit of work, the problem-sized world: a set of files in a public Git repository that keeps one open problem together with its established results, failed routes, experiments, literature and exact open obligations. A short paper compresses its frontier for an expert; a long record keeps the complete arguments; Lean checks the formal proofs; Comparator checks that each selected proof establishes exactly its separately written statement; and a claim registry records the wording, evidence class and limits of every public claim. Any model, provider or person can resume from the record, and accepted work returns to it with credit. A public Lean repository maintains eight open Erdős problems this way. Its results include a kernel-checked irrationality criterion for Problem 257 that admits supports with divergent reciprocal sum, and a formal refutation of the Formal Conjectures statement of Problem 1041, built on a counterexample posted by the forum contributor ani; Formal Conjectures now records the answer as False and links the proof. A comparison with Prove2Me, a hosted platform for stating and proving Lean theorems, shows how the two designs compose.

Contribution and scope

Design. The problem-sized world as the persistent unit of AI-assisted research; a path from proof to public claim that separates what Lean settles, what Comparator settles and what a mathematician judges; and a contribution cycle that a person or an agent can run from a public clone. **Instance.** Eight open Erdős problems in one Lean repository: sixteen problem papers totalling 773 pages, 1,823 Lean modules, and 674 results asserted in the papers, of which 671 carry a Lean declaration and 633 have been compared by Comparator. **Scope.** For Problem 1041 the Formal Conjectures statement is refuted and its match with the 1958 wording awaits independent review, so the claim registry keeps all eight original problems open. The system leaves novelty and significance to experts.

1 What AI-assisted mathematics loses

Mathematicians writing in August and September 2026 describe one shift from several sides. Kra writes that the machines are producing solutions “faster than the mathematical community can read them, much less digest them” [1]. Cohn finds current models “much better at solving mathematical problems than at communicating the solutions well to humans”, and calls the unexplained remainder technical debt: a correct result that someone else must later pay to understand [2]. Tao argues that AI tools flatten the difficulty landscape a field uses to choose its

**Authorship and AI use.* Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

next questions, that withheld negative results and processes hide where the new frontier lies, and asks for analysis that “identifies insights from the solution process” [3]. A declaration first signed by twenty-five Fields Medallists names what rushed announcements lose: the writeup, the isolation of new methods and ideas, and the citation of earlier work [4]. Sanderson compresses the point: “every AI-generated proof is born an unsolved exposition problem” [5].

Each of these describes a loss of state. A proof found in a session arrives without the session: the routes that failed, the computations that chose the route, the literature it relied on and the precise statement that remains open. The next person, or the next model, begins again from the problem statement. As producing results becomes easier, helping other people understand and use them becomes the scarce part of the work.

This paper describes a research environment built around a persistent unit of work, the *problem-sized world*: one open problem together with everything learned about it. A world is written for two readers at once. Its short paper compresses the frontier so that an expert can inspect the underlying mathematical ideas without reconstructing the whole search; its long record keeps the complete arguments for whoever continues. Any model, provider or person can resume from the world, so the cumulative mathematical work stays available to be reasoned over and built on, and a stronger model starts from the recorded frontier.

The paper makes four contributions: the problem-sized world as the persistent unit of AI-assisted research (Section 2); a path from proof to public claim that keeps three questions apart, namely whether Lean accepts a proof, whether the proved statement is exactly the stated one, and whether that statement means what the problem means (Section 3); a contribution cycle, published as agent skills in the public repository, through which a person or an agent can take up an open obligation and return the result with credit (Section 4); and a public instance on eight open Erdős problems (Section 5), compared with Prove2Me and related systems (Section 7).

2 A problem-sized world

A problem-sized world is a set of files in a public Git repository, organised around the mathematics. Table 1 lists its parts.

The short paper is the first read. It compresses the whole record, so an expert can see the frontier of ideas in 12 to 28 pages and decide whether the problem deserves their time. Its margin marks show, result by result, whether belief rests on Lean, on Lean and Comparator, or on the argument printed in the paper; a dagger on a Lean mark flags a named input the formal proof assumes. The long record holds what the essays above find missing from announced results: the solution process, the routes that failed and why, and the credit owed to earlier work. The difficulty landscape Tao describes [3] is kept explicitly, as the set of results, obstructions and open obligations around the endpoint.

2.1 Failed routes and open obligations

The records treat negative results as outputs. A failed attempt shows that one search path did not close. A counterexample refutes a conjecture on its exact domain. A Lean no-go theorem rules out a class of strategies under explicit hypotheses. The last two prevent repeated work and show which new ingredient an endpoint needs. For Problem 257, a proposed estimate said that the greedy test ruling out a rational number with denominator Q as the sum of an infinite subseries of $\sum_n 1/(2^n - 1)$ stops within about $2 \log_2 Q - 3.3$ steps; an [exact rational probe](#) first rules out 189/388 at step 17. The record keeps both that this defeats the estimate and that it leaves the problem itself untouched.

Part	What it holds	Written for
Short paper	The problem and its status, the strongest results and the mechanism behind them, relation to prior work, and the exact point where the argument stops	An expert deciding whether to engage
Long record	Complete arguments, failed routes, experiments with their finite domains, corrected claims, and attribution for every borrowed idea	Whoever continues the work
Lean source	Formal statements and proofs checked by a pinned Lean [7] kernel	Anyone who needs certainty about a formal statement
Evidence marks	Beside each result, “Lean” links the declaration that proves it, directly or through its evidence record, and “Comparator” links its replay receipt; an unmarked result rests on the paper’s own argument	A reader checking one statement
Claim registry	The registered wording of each public claim, with its evidence class, the bounded domain of each finite instance and the open statements it approaches	Release checks and anyone citing a claim
Experiments	Code, inputs, finite domain, output and stated limits	Route selection and replay

Table 1: The parts of one problem-sized world.

Open obligations are stated as exactly as results. Problem 249 asks whether $S = \sum_{n \geq 1} \varphi(n)/2^n$ is irrational, where φ is Euler’s totient function [6]. The development expresses irrationality through finite certificates along $H_t = \text{lcm}(1, \dots, t)$. Writing $\text{Cert}(t)$ for the existence of a finite computation showing that a scaled tail difference of S at H_t is not an integer, Lean checks

$$\forall t \leq 82, \quad \text{Cert}(t),$$

and Lean also proves that irrationality is equivalent to the unbounded supply

$$\forall T, \exists t > T, \quad \text{Cert}(t).$$

The finite theorem makes no $t = 83$ or cofinal claim. What remains open is the unbounded supply itself, which is the irrationality of S in equivalent form, and the record states it in exactly that form.

3 From a proof to a public claim

A proof becomes a public claim by crossing three questions that the word “verified” tends to merge. Table 2 keeps them apart.

Lean verifies that a proof establishes the formal statement written in the source; it does not verify whether that statement captures the intended mathematics or whether the paper describes it well. The gap is measurable. Feng et al. had mathematicians grade 200 candidate solutions to Erdős problems produced by Aletheia, an agent built on Gemini Deep Think: 63 were technically correct and 13 meaningfully correct, and the other 50 answered a reading of the problem that missed Erdős’s intent [8].

Comparator, a checker the Lean FRO built for judging machine-written proofs [9], answers the second question for selected statements. A trusted challenge module restates each statement

Question	Answered by	Recorded in
Does the proof establish this formal statement?	The pinned Lean kernel, in continuous integration	Lean source and build receipts
Is the proved statement exactly the separately stated one, using only permitted axioms?	Comparator, with the Lean kernel and an independent checker	The Comparator ledger and evidence records
Does the formal statement say what the problem says, and does the result matter?	A mathematician	Claim wording in the registry, and review events when they occur

Table 2: Three questions a public mathematical claim must cross.

without its proof; the proof must be a term of exactly that type, using only the permitted axioms, and the Lean kernel must accept it. The corpus replay also passes every proof through nanoda, an independent kernel [10], and continuous integration requires a deliberately altered statement to fail, so the harness cannot pass vacuously. Comparator cannot decide whether a statement is the right translation of the problem. It guarantees that the statement a reader inspects is the statement that was proved, so “Comparator-checked” is accurate where “independently verified” would overclaim.

The third question stays with people. The claim registry records the wording of each public claim with its evidence class, the bounded domain of each finite instance and the open statements it approaches, and the papers, README and generated views may say only what the registry allows. In this repository the maintainer is responsible for that wording, and no independent mathematical review is yet recorded. A release checker keeps those surfaces consistent with the registry and the Lean source, and requires a registered limitation to stay visible wherever its claim appears.

That requirement comes from a test in which a limitation escaped. One deliberately false README edit changed a clause saying that the finite cases of Problem 249 did not supply the open requirement into one saying that they completed it. Lean was untouched, and the checker passed because that relationship had not been registered. After registration, the post-repair witness accepts the current README and rejects a test copy containing the false clause. A small historical study tested the checker with deliberately false edits: nine of the ten edits were rejected. One escaped, for the reason just given. The edits were authored by the checker’s author. The original run logs were not retained. The other nine edits were not rerun against the extended checklist. The evidence marks a coverage boundary, not a reliability score.

The workflow does not technically force a second independent mathematician: one maintainer can edit a Lean statement, its record and its prose together so that every comparison agrees with the same mistake. Independent review is recorded as an external event when it happens.

4 The contribution cycle

Someone outside the original research can contribute a correction or take an argument further, and that work returns to the same record for that problem. The public repository publishes this research, checking and revision cycle as thirteen reusable agent skills that install into common agent harnesses. Each skill owns one job: its starting state, the changes it may make, the evidence it must return, its stopping condition and the next owner. Table 3 shows the ordinary path.

Step	Public skill	What it returns
Orient	explain-public-system, explore-the-corpus	An account of the record adapted to its reader, citing the evidence behind each statement
Work	mine-open-problem	The smallest evidence-bearing change: a theorem, a counterexample, a no-go result or a diagnosed failure
Validate	lean-concurrent-validation, land-lean-proofs	One Lean verdict per distinct request, and every paper that states the result linked to its declaration
Propagate	propagate-research-consequences	A recorded disposition for every affected import, claim, paper and open obligation
Package	erdos-research-return, public-mathematical-writing	A provenance-preserving return, with its prose held to the evidence
Propose	submit-pull-request	A pull request from a named starting commit
Adopt	Maintainer review under the credit policy	Reconciliation with the current tree and a public credit receipt

Table 3: The contribution cycle as published in the public repository. Four further skills coordinate discovery and stewardship, admit a new problem world, maintain the clone’s own tooling and install the set. A cycle closes when the change has evidence and every consequence has a recorded disposition; the open problem closes only with a proof of its original statement.

Two steps carry most of the discipline. Propagation starts from the changed object and lists every reverse import, claim record, paper, experiment and open obligation the change may affect. Each must be updated, verified unchanged, deferred with a reason or marked out of scope, and an empty search never counts as evidence of no consequence. Adoption replays the contributor’s original change from its named starting commit before reconciling it with the current tree, so the original work and any conflict resolution are credited separately. Orientation needs no knowledge of the file layout: `explain-public-system` adapts its account to a lay reader, a mathematician, a formaliser, a compute contributor or an infrastructure contributor. The [companion contribution protocol](#) gives the full sequence.

Discovery and stewardship. Long-running work uses two coupled roles. A discovery pass stays close to the frontier: it runs discriminating computations, attempts proofs and returns results. A stewardship pass reads each new result against the whole corpus and decides what it establishes, whether it is routine or strong, where it belongs in a paper, and which obligation deserves the next unit of work. One agent may play both roles in sequence. Neither inherits the other’s authority, and both wait while the frontier and its consumers are unchanged.

Credit. The `credit policy` lets an idea be credited separately from its proof, its Lean formalisation and its explanation, the roles Kra asks journals to distinguish [1], and credits software as a role of its own. A first return can be a plain-language research-progress issue or email: no clone, proof, code or receipt schema is required. Accepted work receives a public receipt tied to exact artifacts. Acceptance does not establish theorem status, novelty, or release inclusion. Corrections append to the history, and an earlier contributor’s record stays in it.

4.1 Worked example: Problem 1041

Problem 1041 shows the cycle end to end, with the decisive idea arriving from outside. It asks whether, for a monic polynomial f with all roots in the open unit disc, two roots can always be joined inside the lemniscate $\{|f| < 1\}$ by a path of length less than 2 [11, 12]. On 7 September 2026 the erdosproblems.com contributor ani posted a degree-seven counterexample, found, as the post says, with the help of GPT-6 [13], and the record took the construction as an attributed input five days later. Lean first checked the total-variation form. On 23 September it proved that for that polynomial every preconnected subset of the strict lemniscate containing two distinct roots has one-dimensional Hausdorff measure greater than 2, which refutes the exact path-image-length statement in Google DeepMind’s Formal Conjectures repository [14]. The same day, the Formal Conjectures maintainers merged a change that marks the problem solved with answer False and links this proof [15]. The short paper and long record carry the proof with its Lean mark and credit ani for the construction. One judgement remains open: whether the formal statement matches the 1958 wording [12, Problem 5, p. 139]. The record says so beside the result.

5 Eight problems in one repository

The [public repository](#) maintains eight open Erdős problems [16] as problem-sized worlds. Its README leads with Problem 257, which asks whether $\sum_{a \in A} (2^a - 1)^{-1}$ is irrational for every infinite set A of positive integers. For a set A of positive integers and a finite nonempty set P of primes, let $h(a) = \prod_{p \in P} p^{v_p(a)}$ be the P -part of a , where $v_p(a)$ is the exponent of p in a . If

$$\sum_{a \in A} \frac{h(a)}{a (2^{h(a)} - 1)} < \infty,$$

then $\sum_{a \in B} (b^a - 1)^{-1}$ is irrational for every integer $b \geq 2$ and every infinite $B \subseteq A$. The condition admits sets A whose reciprocal sum diverges, and Lean checks the criterion. Table 4 gives one result for each problem and the point where it stops.

The repository holds 1,823 Lean modules under a pinned toolchain, sixteen problem papers totalling 773 pages, a cross-problem synthesis paper and three systems papers. The papers assert 674 results. Of these, 671 carry a Lean mark, linking a declaration that proves the result exactly, in a stronger form or from a named input, and 633 have been compared by Comparator; every receipt in the pinned corpus replay was accepted by both the Lean kernel and nanoda. The claim registry holds 150 public claims in seven statuses, from proved here and formalised here to conditional reduction and open, with 19 open propositions stated exactly. Continuous integration builds the supported Lean roots, a coverage build compiles every module a paper cites, and the release checker runs 15,666 checks over claims, papers, generated views and licences. A fresh clone needs no private file, and no public theorem depends on an unpublished lemma.

6 How the records are produced

One person has built, over the past year, the private environment that produces the records, and runs agents from several model providers in it, including Anthropic’s Claude and OpenAI’s GPT models, concurrently on one shared file system. Five design choices carry most of the weight.

Shared state on disk. Durable state lives in files: sources, structured records, append-only ledgers, validation receipts and authored explanations. A change exists once its file is written and its checker accepts it, so work resumes across sessions, models and providers.

Problem	A result to start with	Where it stops
#68, factorial denominator	Lean checks that the logarithm of the uncleared common denominator grows at least like $N^{3/2} \log N$, and that irrationality is equivalent to cofinally many non-unit factorial carries	Those carries are not produced
#243, Sylvester-tail rigidity	Lean checks irrationality of $\sum 1/a_n$ under the exact cubic rate $a_n^2/a_{n+1} = 1 + 3/n + o(n^{-3})$	The unrestricted Sylvester-tail question
#249, binary totient series	In every base k , the totient sections $\varphi(k^j n + r)$ with $j \leq e$ span dimension $k^e + 1$; certificates for every $t \leq 82$	Certificates for arbitrarily large t
#251, prime-gap dyadic series	Lean checks a synthetic prime-gap countermodel; the paper proves a sparse-perturbation obstruction	An argument specific to actual prime gaps
#257, infinite exponent supports	The weighted-support criterion above, at every integer base	Arbitrary infinite supports
#269, three-prime running LCMs	Both two-prime sums are transcendental, by a cited Hecke–Mahler theorem, the repeated-sum case first posted by Fan [17]; Lean checks the formulas and the conditional transfer	The three-prime case
#1041, lemniscate connections	The Formal Conjectures statement is refuted with ani’s example	Match with the 1958 wording
#1049, rational-base Lambert series	Lean checks irrationality in Zudilin’s rational-base region and exact Hankel orders	Base 3/2 and all rational bases

Table 4: One result per problem, with its stopping point. The papers state each result in full with its hypotheses and sources.

Bounded context. Before an agent opens a large source, a router compiles a packet conditioned on its task: the endpoint, its status and claim ceiling, the strongest nearby results, known falsifiers and literature, each labelled by evidence class, with a list of what was omitted and the command that restores it. A packet that would exceed its budget reports the overflow and stops.

Claims before writes. An agent claims exact paths for a bounded lease before changing them. Overlapping claims wait or coordinate, expired leases lose authority, and parallel work meets at a fan-in step where results are compared and validated.

Single-flight validation. Lean builds pass through a semantic single-flight queue keyed by source, targets, toolchain and dependency lock. Equivalent requests share one build. An agent that cannot get the build slot receives a deferral, recorded separately from theorem failures, and continues with independent work.

Route repair. A failure in navigation, scheduling, experimentation or validation can change a route, skill or check for later agents through a guarded proposal. It never changes the status of a mathematical claim.

The public repository replays without the private environment. The private environment explains how the records were produced; the public evidence is what supports each claim.

7 Relation to Prove2Me and other systems

Prove2Me [18] is the closest published system. It is a hosted platform on which anyone with a coding agent states Lean theorems and submits proofs. Each statement is immutable and may carry several proofs; the server accepts a proof only when its type is exactly the target’s, with no sorry and only whitelisted axioms, and a disproof proves the negation. Missions on papers, textbooks and open problems confine human audit to a goal, its definitions and milestone lemmas, aided by a blind agent read-back of each Lean statement, and a proof sketch may import open statements for other agents to close. Prove2Me began as a class game at Columbia in spring 2026 and its paper appeared on 28 August 2026; the first public version of this paper, with its claim registry and release checks, appeared in July 2026, and the two designs developed independently. Prove2Me is the published reference for hosted statement and proof separation, audited mission cores and read-back auditing. Table 5 sets the two side by side.

	Prove2Me	Problem-sized worlds
Unit of work	One immutable statement or proof within a mission	One open problem with its whole record
What persists	Statements, proofs, sketches, explanations, milestones and discussion; disproofs and deprecations stay visible	Short and long papers, failed routes, experiments, typed claims and open obligations beside the Lean source
Exact-statement check	Every submission, on the server	Selected statements, through Comparator, with a second kernel in the corpus replay
Statement fidelity	A captain audits each mission core with a blind read-back; a moderator approves public missions	The maintainer is responsible for the wording of each public claim in the claim registry; no independent review is recorded yet
Exposition	A description for each statement and an explanation for each proof	A short paper and a long record for each problem, with evidence marks at each result
Credit	Permanent names, trust scores, leaderboards and import citations	Receipts by contribution role, and attribution for outside ideas
Contribution path	Web platform and API, with an account	Git clone and thirteen agent skills, a pull request, an issue or an email
Scale, 24 September 2026	373 missions, 88,246 theorems, 722 users	Eight problems, 1,823 Lean modules, one maintainer

Table 5: Prove2Me and problem-sized worlds, from Prove2Me’s paper and public pages and from this repository, on 24 September 2026.

The two compose. Prove2Me contributes hosting, importable statements, per-submission checking and a contributor base; a world contributes the problem-level context around a statement: its papers, failed routes, obstructions and open obligations. The Problem 1041 package

from this repository is hosted on Prove2Me, where its 21 theorems have accepted proofs and an is credited separately for the construction.

Other systems. Feng et al. ran an agent over the 700 Erdős problems then listed as open and published graded outcomes [8], and propose reporting AI-assisted results on at least two axes, autonomy and significance, with significance judged only by mathematicians expert in the area [19]; the claim registry records the evidence behind a result and leaves its significance to experts in the same way. Tao’s community wiki listed AI contributions to the Erdős problems, failed attempts included, until its updates stopped on 30 June 2026 [20]; a registered claim, with its evidence class and limits, is the kind of record such a list can cite. Formal Conjectures [14] turns open problems into formal statements that automated systems can target. Ringer distinguishes mathematics as a process from mathematics as a benchmark [21]; a world keeps the process around such a statement. Blueprints connect informal proof plans to Lean declarations [22, 23], and provers such as ReProver, built on the LeanDojo toolkit, and OpenProver [24, 25] search for proofs; a world can use a blueprint as its dependency map and any such prover as a source of candidate proofs. Li et al. published a human-in-the-loop protocol for research mathematics that holds generation and validation apart without a proof assistant [26].

8 Limits, and what stronger models change

All eight problems remain open in the claim registry, and for Problem 1041 the match between the refuted formal statement and the 1958 wording awaits independent review. The system records evidence classes and review events; novelty and significance are judgements it leaves to experts, as Feng et al. also propose [19]. One maintainer can make a coordinated mistake that every structural check accepts. By 24 September 2026 no outside contributor had opened a pull request or issue in the repository; its outside record is in Formal Conjectures, whose maintainers have merged four of its contributions, and on Prove2Me, which hosts its Problem 1041 package with accepted proofs. A record also leaves several of the concerns in Section 1 where it finds them: it trains no one, it restores no signal of individual expertise [27], and it answers neither the ethical objections some mathematicians raise to AI-assisted mathematics [28] nor the warning of forty-two Fellows and Foreign Members of the Royal Society about catastrophic risk from AI [29]. The private production environment is described here and is not distributed.

The same design serves stronger models. A stronger model starts from the current frontier with the failed routes already marked, and it can reorganise, re-explain and extend the record as well as the mathematics. The public interfaces admit any producer: an outside contributor can attach any agent runner to the clone, and a laboratory with many frontier models could replace the private production environment while keeping the same evidence boundary.

Gowers suggests a well-designed database of what is known, probably built with AI help, and Koukoulopoulos a public facility in which coordinated agents explore a research programme divided into subproblems under researchers’ direction [30, 31]. A problem-sized world supplies the shared record both proposals need, at the scale of eight problems. Antieau writes that texts generated largely by a language model deserve a home other than the arXiv [32]; a world gives its long records one beside the proofs they explain. The same structure suits a single landmark result: a short paper explaining its core ideas and relation to prior work, connected to the full argument, the relevant formal checks and the questions it opens, with outside corrections and extensions returning to the same record. The design asks three things of a field: stable result identities, a checker where one exists, and an explicit boundary on public claims. The next measurement is comparative: whether agents given a world choose better frontiers, repeat fewer dead ends and reach useful obstructions sooner than agents given the same mathematics as prose.

9 Conclusion

A problem-sized world keeps the record around a proof that lets other people understand it, check it, credit it and build on it: its frontier, failed routes, formal checks, attribution and exact open statements. The eight worlds in the public repository show that one person can keep that record persistent, checkable and open to contribution across several hard problems. The record is built for models stronger than the ones that produced it: each starts from the recorded frontier with the failed routes marked, and Section 8 names the measurement that tests whether this makes it more effective.

A Inspection routes and reproducibility

Question	Start here
How does the repository fit together?	docs/ARCHITECTURE.md
What is proved and what is open?	docs/RESULTS.md and docs/claims.json
Where is the checked mathematics?	lean/ErDOS249257.lean and lean/ErDOSProblems.lean
What does Comparator check?	docs/EXTERNAL_VERIFICATION.md and verification/comparator.json
Which checks gate a release?	scripts/check_release.py and .github/workflows/lean.yml
How can work return with credit?	CONTRIBUTING.md , credit policy and architecture contributions
Which agent skills exist?	skills/README.md

Table 6: Entry points for inspection. They lead to the authority-bearing files and add no authority of their own.

A fresh clone reproduces the control card and the structural checks without Lean:

```
python3 scripts/proof_cockpit.py --format card
python3 scripts/proof_cockpit.py --check
python3 scripts/check_release.py
```

Formal authority begins with the pinned lake build named by the card. The paper inventory, `docs/publication_contract.json`, records source and PDF hashes and validation commands, and the evidence for the checker study in Section 3 is recorded in `docs/publication_evidence.json`.

References

- [1] B. Kra, *Deep theorems were scarce and difficult and so became an effective mechanism to identify deep thought. AI has broken this system*, guest post on *What's new*, 13 September 2026, [blog post](#).
- [2] H. Cohn, *The technical debt of AI-generated mathematics*, guest post on *What's new*, 15 September 2026, [blog post](#).
- [3] T. Tao, thread on mining open problems, Math-sodon, 8 September 2026, [thread](#).
- [4] Twenty-five Fields Medallists, *A severe misalignment of AI in mathematics*, declaration posted on *What's new*, 11 September 2026, [blog post](#).
- [5] G. Sanderson, *If math is more than proof, we need to better celebrate the rest of it*, guest post on *What's new*, 18 September 2026, [blog post](#).
- [6] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, *Monographies de L'Enseignement Mathématique* 28, 1980, p. 61.
- [7] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction*,

- CADE 28, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, [DOI](#).
- [8] T. Feng, T. Trinh, G. Bingham, et al., *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*, 2026, [arXiv](#).
- [9] Lean FRO, *Comparator*, 2025, [GitHub](#).
- [10] ammkrrn, *nanoda_lib*, an independent type checker for Lean 4, [GitHub](#).
- [11] T. F. Bloom, *Erdős Problem #1041*, [erdosproblems.com](#), accessed September 2026.
- [12] P. Erdős, F. Herzog and G. Piranian, *Metric properties of polynomials*, *J. Analyse Math.* 6 (1958), 125–148, [DOI](#).
- [13] ani, post in the Problem 1041 discussion thread, 7 September 2026, [erdosproblems.com forum](#).
- [14] Google DeepMind, *Formal Conjectures*, [GitHub](#), accessed September 2026.
- [15] Formal Conjectures, pull request 6505, *Erdős 1041: mark solved with answer(False) and link a formal proof*, merged 23 September 2026, [GitHub](#).
- [16] T. F. Bloom, *Erdős problems*, [erdosproblems.com](#), accessed September 2026.
- [17] S. Fan, comment on Erdős Problem #269, [erdosproblems.com forum](#), 26 June 2026, [forum post](#).
- [18] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng, *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*, 2026, [arXiv](#).
- [19] T. Feng, T. H. Trinh, G. Bingham, et al., *Towards Autonomous Mathematics Research*, 2026, [arXiv](#).
- [20] T. Tao, *AI contributions to Erdős problems*, [GitHub](#), accessed September 2026.
- [21] T. Ringer, *Becoming a benchmark*, guest post on *What’s new*, 17 September 2026, [blog post](#).
- [22] P. Massot, *leanblueprint*, *plasTeX* plugin for Lean formalisation blueprints, 2020, [software repository](#).
- [23] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *Lean-Architect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, *LIPIcs* 382, 2026, pp. 25:1–25:16.
- [24] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, *NeurIPS* 2023.
- [25] M. Kripner and M. Straka, *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*, 2026, [arXiv](#).
- [26] C. Li, Z. Lai, D. An, J. Hu, and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, 2025, [arXiv:2512.09443v2](#).
- [27] D. Litt, *A beginning for mathematics*, 13 September 2026, [blog post](#).
- [28] T. Chu, *The AI dissenter viewpoint*, *Proofs and Prompts*, 9 August 2026, [blog post](#).
- [29] B. Green and forty-one other Fellows and Foreign Members of the Royal Society, *Open letter to Sir Paul Nurse, President of the Royal Society*, *Proofs and Prompts*, 17 September 2026, [blog post](#).
- [30] W. T. Gowers, *Why I didn’t sign the Fields medallists’ letter*, 17 September 2026, [blog post](#).
- [31] D. Koukoulopoulos, *A CERN for AI-assisted science?*, guest post on *What’s new*, 17 September 2026, [blog post](#).
- [32] B. Antieau, *Fast math/slow math*, 15 September 2026, [blog post](#).